

hardcaml_asic: Concept Note

A proposal for a Hardcaml ASIC technology and flow abstraction layer

Purpose

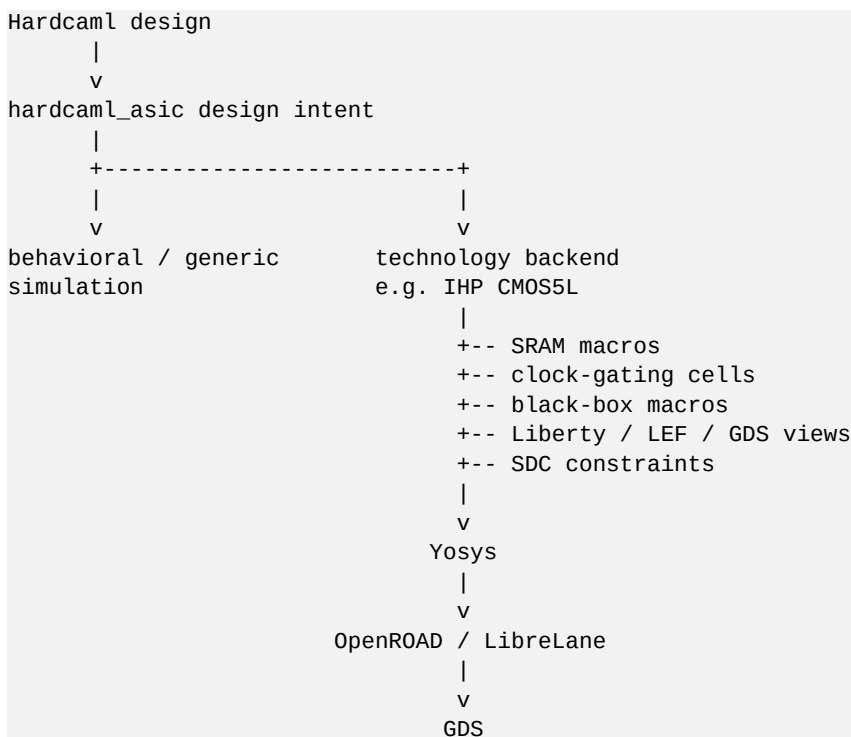
hardcaml_asic would be a Hardcaml library for expressing ASIC-specific design intent and connecting Hardcaml-generated RTL to ASIC technology primitives and implementation flows.

The library should not be a catalog of individual NAND gates, flip-flops, or drive-strength variants. Generic logic synthesis already handles that job. Instead, it should focus on resources and constraints that do not map cleanly from generic RTL: SRAM macros, clock-gating cells, black-box macros, timing constraints, technology selection, and physical-design flow integration.

The Jane Street protocol emulator ASIC competition is a useful forcing function and reference application: it provides a concrete design that must move from Hardcaml RTL into an ASIC flow, exposing the exact places where FPGA-centric abstractions stop being sufficient.

Core Design Principle

The intended abstraction boundary is:



A Hardcaml design should request a resource by intent, rather than naming a particular foundry macro directly.

For example:

```
Sram.create
~width:16
~depth:256
~read_latency:1
...
```

rather than embedding something like:

```
Instantiation.create
~name:"some_foundry_specific_256x16_sram"
...
```

The backend decides how the requested resource is implemented.

Proposed Scope

1. SRAM and Register-File Abstraction

This should be the first feature and likely the most immediately valuable one.

ASIC SRAMs are physical macros. They usually come with multiple required views: a logical or simulation model, timing data, abstract physical geometry, and final layout geometry. A generic Hardcaml memory description is therefore not always enough to express what should happen during ASIC implementation.

A portable API might resemble:

```
module Sram : sig
  type read_during_write =
    | Read_first
    | Write_first
    | Undefined

  type config =
    { width : int
    ; depth : int
    ; read_latency : int
    ; read_during_write : read_during_write
    }

  val create
    : config
    -> clock:Signal.t
    -> address:Signal.t
    -> write_enable:Signal.t
    -> write_data:Signal.t
    -> Signal.t
end
```

The same design could then use different implementations:

```
Generic simulation      -> behavioral Hardcaml RAM
Generic synthesis       -> inferred memory / flop-based fallback
IHP CMOS5L backend     -> physical SRAM macro
Future SKY130 backend  -> corresponding SKY130 macro
```

This keeps foundry-specific macro names out of the architecture itself.

2. Technology Backend Interface

A small technology interface should own mappings from generic ASIC design intent to process-specific implementations.

Conceptually:

```
module type Technology = sig
  val name : string

  module Sram : Sram_intf.S

  val clock_gate
    : clock:Signal.t
    -> enable:Signal.t
    -> Signal.t

  val tie_high : unit -> Signal.t
  val tie_low : unit -> Signal.t
end
```

Possible package structure:

```
hardcaml_asic/
  technology.ml
  sram.ml
  register_file.ml
  clock_gate.ml
  constraints.ml
  macro.ml
  flow.ml

hardcaml_asic_ihp_cmos5l/
  technology.ml
  sram.ml
  clock_gate.ml
  macros/

hardcaml_asic_sky130/
  ...
```

Process-specific support should ideally live in separate backend packages rather than accumulating conditionals throughout the core library.

3. ASIC Timing Constraints

hardcaml_asic could represent common timing constraints as typed OCaml data and emit SDC.

Example:

```
let constraints =
  [ Clock.create ~signal:"clk" ~period_ns:20.
  ; Input_delay.create ...
  ; Output_delay.create ...
  ]
;;

Constraints.write_sdc constraints "design.sdc"
```

An initial implementation only needs a useful subset:

- create_clock

- set_clock_uncertainty
- set_input_delay
- set_output_delay
- set_false_path
- set_max_delay

The goal is not to reimplement all of SDC. The goal is to keep common design constraints close to the Hardcaml source and generate deterministic flow inputs.

4. Generic Macro / Black-Box Support

SRAM is only one kind of non-generic ASIC resource. A generic macro description could capture the relationship between logical and physical views:

```
Asic.Macro.create
~name
~ports
~simulation_model
~physical:
{ liberty
; lef
; gds
}
```

This would provide one place to register the artifacts required by downstream implementation tools.

Typical macro views are:

```
Verilog model  -> simulation / synthesis black box
Liberty        -> timing and power characterization
LEF            -> abstract dimensions and pin geometry
GDS            -> physical mask geometry
```

5. Physical-Design Flow Integration

The library should orchestrate existing tools rather than replace them.

A possible user experience:

```
dune exec ./bin/build.exe -- gds
```

with the library coordinating:

```
Hardcaml
-> generated RTL
-> generated SDC
-> Yosys
-> synthesized netlist
-> OpenROAD / LibreLane
-> floorplan
-> placement
-> clock-tree synthesis
-> routing
-> GDS
```

It could also parse implementation reports into OCaml values:

```
let result = Asic.Flow.run design in
printf "Area: %.0f um^2\n" result.area;
```

```
printf "WNS: %.3f ns\n" result.wns;
printf "TNS: %.3f ns\n" result.tns;
```

Useful metrics include:

- standard-cell area and count
- utilization
- macro area/utilization
- WNS/TNS
- wirelength
- clock skew
- DRC count
- estimated power

This is analogous to making ASIC implementation reports accessible in the same way FPGA-oriented tooling can expose synthesis and timing information programmatically.

What Should **Not** Be in the Library

The library should generally not expose every standard cell as a Hardcaml primitive:

```
NAND2_X1
NAND2_X2
NAND2_X4
AOI21_X1
INV_X8
...
```

Generic RTL should normally remain generic:

```
Hardcaml Boolean logic
    |
    v
  Yosys
    |
    v
technology-mapped NAND/NOR/AOI/OAI/etc.
```

The synthesis tool needs freedom to select equivalent cells based on timing, area, fanout, and drive strength.

Explicit primitives make sense when the resource has architectural meaning or cannot be safely inferred, such as:

- SRAMs and register files
- integrated clock-gating cells
- special latches or synchronizer cells
- I/O cells
- analog or mixed-signal macros
- hard IP blocks
- process-specific special cells when required by the flow

The API should expose the intent (clock_gate, sram, macro) rather than encouraging the design to depend directly on a specific cell name.

Relationship to the Protocol Emulator Competition

The protocol emulator should act as the reference design and integration test for hardcaml_asic.

```
hardcaml_asic
|
+-- SRAM abstraction
+-- technology backend
+-- SDC generation
+-- macro registration
+-- OpenROAD / LibreLane runner
+-- report parser
|
v
protocol_emulator
|
+-- processor / sequencer
+-- instruction memory
+-- protocol engines
+-- GPIO / external interface
+-- host/configuration interface
```

This is preferable to designing a large generic library in isolation. Every abstraction should be added because the emulator or another real ASIC design needs it.

The strongest initial use case is likely instruction/data memory: the emulator will probably need compact storage, which immediately forces the design to deal with real SRAM macros rather than FPGA block RAM inference.

Recommended MVP Roadmap

v0.1 - SRAM abstraction

Implement one portable SRAM API with:

- behavioral simulation model
- generic fallback implementation
- one physical IHP CMOS5L macro mapping

Success criterion: the same Hardcaml module can simulate normally and elaborate into RTL that uses an ASIC SRAM macro without changing its architectural source.

v0.2 - IHP CMOS5L backend

Create an explicit technology package containing:

- supported SRAM descriptions
- macro names and ports
- required physical/timing collateral references
- any required special-cell wrappers

v0.3 - Constraint generation

Generate a minimal SDC file from typed Hardcaml-side declarations.

v0.4 - Flow orchestration

Provide a reproducible command/API that runs:

```
Hardcaml -> Yosys -> LibreLane/OpenROAD -> GDS
```

Do not hide the underlying tools completely; make commands, logs, and generated artifacts inspectable.

v0.5 - Report parsing

Parse implementation results into machine-readable OCaml structures for timing, area, utilization, DRCs, and related metrics.

v0.6 - Additional ASIC primitives

Add resources only as real designs require them, likely beginning with:

- clock gates
- generic macro registration
- register files
- special I/O or synchronization primitives

v1.0 - Portable ASIC Design Kit

A stable core API with at least one well-supported open PDK backend and a complete example design that goes from Hardcaml source to verified physical implementation.

Suggested Repository Philosophy

A useful guiding analogy is:

```
hardcaml_xilinx
= make FPGA-specific resources pleasant to use from Hardcaml

hardcaml_asic
= make ASIC-specific resources and flows pleasant to use from Hardcaml
```

The library should therefore concentrate on the boundary where generic RTL stops being enough, rather than attempting to become a synthesis or place-and-route system itself.

Three principles should guide development:

1. **Intent over cell names.** Architecture asks for an SRAM or clock gate; the technology backend chooses the implementation.
2. **Wrap established tools.** Yosys, OpenROAD, and LibreLane remain responsible for synthesis and physical design.
3. **Let real designs drive abstractions.** The protocol emulator should be the first integration test, preventing the library from becoming an over-designed framework with no practical consumer.

Questions for Review

The following are the main design questions worth discussing before implementation grows significantly:

4. Should technology backends live in separate opam packages (`hardcaml_asic_ihp_cmos5l`, `hardcaml_asic_sky130`) or as submodules of one package?
5. How much memory behavior should the portable SRAM API promise across technologies, especially read-during-write behavior, byte enables, ports, and latency?
6. Should the library own tool invocation directly, or expose a declarative project description consumed by Dune/Nix/external scripts?
7. How tightly should timing constraints be coupled to Hardcaml Signal.t objects versus names/hierarchical paths in the generated RTL?
8. How much physical information should enter the OCaml API? For example, should macro placement/floorplanning hints eventually be representable?
9. Is a generic Macro abstraction useful early, or should the project begin narrowly with SRAM and expand only once another concrete macro type appears?
10. What is the cleanest way to verify that simulation semantics match the selected physical SRAM macro semantics?

Recommended Starting Point

Build the smallest possible end-to-end demonstration:

```
Hardcaml test design
-> Hardcaml_asic.Sram.create
-> IHP CMOS5L SRAM mapping
-> generated Verilog + SDC
-> Yosys
-> LibreLane/OpenROAD
-> placed/routed design
-> timing + area report
```

Once that path works, use it inside the protocol emulator and add abstractions only when the design encounters a new ASIC-specific requirement.

That would test whether `hardcaml_asic` is a genuinely useful library boundary rather than merely a collection of wrappers.